Updating your setup

NSDF-Data student guide 7 — Windows edition

Version 1 · 12 October 2026

Routine maintenance

Do this regularly, not once. The project keeps changing: new releases bring new code and guides, and sometimes a new version of Python or of the packages the notebooks use. If your setup is behind, things can fail in confusing ways. Follow this guide **whenever the project lead announces a new release**, **whenever the environment check warns you**, and **when you start a new piece of work after a long break**. It takes about 10 minutes, or about 20 if Python itself changed.

Tip

Who this is for. You have finished the setup guide and the first analysis guide for Windows. Nothing here deletes your notebooks or your work. If you are not sure about a step, stop and ask the project lead.

Before you start

- **Time:** about 10 minutes (about 20 if Python changed).
- **You need:** the setup guide finished, and your own work saved in Git (see Step 7 of the first analysis guide). Step 2 below checks.
- **Close Jupyter first:** in JupyterLab choose **File**, then **Shut Down**, and close the terminal that was running it. Updating files while Jupyter is open can fail, especially on Windows.
- **Where to find out what changed:** the release notes. The project lead also writes to you when a release needs something from you: look for the words “Students, please”.

Words you will meet

Word	What it means here
Release	A numbered version of the whole project, such as 0.5.0. The number is in the file `VERSION`.
Environment	The sealed box of programs with fixed versions that you made in the setup guide, called `darkmatter_cli_env`.
Pinned	A package listed in `requirements.txt` with an exact version, such as `numpy==2.2.6`.

Word	What it means here
Merge	Bring the newest shared changes of the project (the branch `develop`) into your own branch.

Step 1. Check your setup

Open a **Miniforge Prompt** (click **Start**, type `Miniforge Prompt`, press **Enter**), and type these lines, one at a time, pressing Enter after each:

```
conda activate darkmatter_cli_env
cd %USERPROFILE%\Research\NSDF-Data\python
python check_environment.py
```

What these lines do: the first switches your environment on, the second moves you into the project's `python` folder, and the third runs the project's environment check.

Checkpoint

If it says `Environment OK`, your environment is current: go to Step 2 anyway if a new release was announced, otherwise you are done. If it says `WARNING`, it lists what differs. A line about **Python** means Step 4. A line about a **package** means Step 3. Do Step 2 first in both cases.

Step 2. Bring the newest project changes into your branch

Your branch is your own copy of the project. The newest shared changes are on the branch `develop`. Type:

```
cd %USERPROFILE%\Research\NSDF-Data
git branch --show-current
git status
```

Checkpoint

`git branch --show-current` prints `student-yourname` (your own branch), and `git status` says `nothing to commit` (or lists only files you have not added, such as your own notebook folder). If it lists changed files you want to keep, **save them first**: commit them as in Step 7 of the first analysis guide, then come back. If the branch is `develop` or `master`, stop and ask the project lead.

Now bring the changes in:

```
git fetch origin
git merge origin/develop
```

What these lines do: `git fetch` downloads the newest changes without touching your files, and `git merge` combines them with your branch. It prints `Already up to date.` or a list of the files that changed.

Careful

If git says `CONFLICT`, stop. Do not edit anything and do not try to fix it: copy the message and send it to the project lead.

Step 3. If packages changed: install the project's list again

Do this when the check named a package. Type:

```
conda activate darkmatter_cli_env
cd %USERPROFILE%\Research\NSDF-Data
python -m pip install -r requirements.txt
```

What these lines do: the last one reads `requirements.txt` and installs every package on the list at the exact version given, replacing any that differ. It prints a lot of text and can take a few minutes.

Then run the check again (Step 1). It should say `Environment OK`.

Step 4. If Python changed: make a new environment

Do this when the check says your Python is not the version the guides use. Python cannot be safely changed inside an existing environment, so you make a new one and keep the old one until the new one works. Type each line and press Enter:

```
conda deactivate
conda rename -n darkmatter_cli_env darkmatter_cli_env_old
conda create -n darkmatter_cli_env python=3.12 pip -y
conda activate darkmatter_cli_env
cd %USERPROFILE%\Research\NSDF-Data
python -m pip install -r requirements.txt
python -m ipykernel install --user --name darkmatter_cli_env
```

What these lines do

Line	What it does
`conda deactivate`	Switches your current environment off.
`conda rename`	Gives your old environment the new name `darkmatter_cli_env_old`. Nothing is deleted.
`conda create`	Makes a new, empty environment called `darkmatter_cli_env`, with Python 3.12 and `pip`.
`conda activate`	Switches the new environment on.
`cd`	Moves you into the `NSDF-Data` folder, where `requirements.txt` is.
`pip install -r requirements.txt`	Installs the project's list of packages. It takes a few minutes.
`ipykernel install`	Tells Jupyter about the new environment, replacing the old entry of the same name.

Careful

Anything you installed yourself into the old environment is not in the new one. To see what it had, type `conda activate darkmatter_cli_env_old` and then `python -m pip freeze`. When the new environment works (Step 5), you can free the disk space with `conda env remove -n darkmatter_cli_env_old`, but there is no hurry.

Step 5. Check that everything works

```
conda activate darkmatter_cli_env
cd %USERPROFILE%\Research\NSDF-Data\python
python check_environment.py
python -m pytest -q
```

Checkpoint

The check says `Environment OK` and `pytest` ends with `62 passed`. If not, do not guess: copy the output and send it to the project lead.

Step 6. Start Jupyter again

```
conda activate darkmatter_cli_env
cd %USERPROFILE%\Research\NSDF-Data\data_analysis\R76\student
cd yourname
jupyter lab
```

Use the same `yourname` as in your branch name. If Jupyter asks for a kernel, choose **darkmatter_cli_env**. Open your notebook and run the first cell: it should print `Environment OK` and then `(1517, 4096)`.

You are done when

- ☐ `python check_environment.py` (in the `python` folder, with the environment on) says `Environment OK`.
- ☐ `git status` in the `NSDF-Data` folder shows your own branch and nothing you want to keep is unsaved.
- ☐ `python -m pytest -q` in the `python` folder ends with `62 passed`.
- ☐ `jupyter kernelspec list` lists `darkmatter_cli_env`.
- ☐ Your notebook's first cell prints `Environment OK`.

If something goes wrong

What you see	What to do
`python: can't open file 'check_environment.py'` or `No such file`	You are not in the `python` folder. Run the `cd` line of Step 1, and make sure Step 2 finished (the file arrives with the newest changes).
`conda rename` says it is not a command	Your conda is old. Ask the project lead.

What you see	What to do
`CondaError` or `EnvironmentNameNotFound` on `conda rename`	The environment `darkmatter_cli_env` may not exist under that name. Type `conda env list` and send the list to the project lead.
git says `CONFLICT`	Stop and send the message to the project lead. Do not edit the files.
The kernel is missing in Jupyter	In the environment, run the last line of Step 4 again, then restart Jupyter.

Where to get help

Anthony Villano, project lead: anthony.villano@ucdenver.edu

When you write, include the step number, the command you typed, and the last few lines the computer printed. A screenshot helps. Nobody will be annoyed by a question about a step that did not work.

Session Info

Guide version 1, 12 October 2026, Windows edition. Written for beginners; please tell the project lead where a step was unclear, so the guide can be fixed.

Key links

- Release notes: https://github.com/villano-lab/NSDF-Data/releases
- Repository: https://github.com/villano-lab/NSDF-Data
- The setup guide and the first analysis guide: https://villano-lab.github.io/NSDF-Data/